

ΟΝΟΜΑΤΕΠΩΝΥΜΟ: ΙΑΣΟΝΑΣ ΤΣΙΟΥΡΑΜΑΝΗΣ

ΑΜ: 1100741

1. ΕΙΣΑΓΩΓΗ

Η προεπεξεργασία δεδομένων (data scaling) αποτελεί ένα από τα σημαντικότερα στάδια σε εφαρμογές μηχανικής μάθησης, επιστημονικών υπολογισμών και ανάλυσης μεγάλων δεδομένων. Όταν ο πίνακας δεδομένων είναι πολύ μεγάλος (δεκάδες Gigabytes) και υπερβαίνει τη διαθέσιμη κύρια μνήμη (RAM) του συστήματος, δημιουργείται η ανάγκη για προσεγγίσεις Out-of-Core επεξεργασίας. Στην παρούσα εργασία ζητείται η υλοποίηση ενός συστήματος παράλληλης προεπεξεργασίας μεγάλων αριθμητικών δεδομένων.

Η δική μας υλοποίηση βασίζεται σε λογική δύο φάσεων:

- **Φάση 1 (Υπολογισμός Στατιστικών):** Διατρέχουμε το αρχείο εισόδου, υπολογίζοντας δυναμικά αθροίσματα, αθροίσματα τετραγώνων, ελάχιστα ($\min$) και μέγιστα ($\max$) για κάθε στήλη. Μετά την ολοκλήρωση της σάρωσης, υπολογίζονται τα τελικά στατιστικά (διακύμανση, τυπική απόκλιση, μέση τιμή) χρησιμοποιώντας τους μαθηματικούς τύπους.
- **Φάση 2 (Εφαρμογή Scaling):** Επαναφέρουμε τον file pointer στην αρχή του αρχείου (μέσω της `fseek/fseeko`), ξαναδιαβάζουμε το αρχείο ανά block, εφαρμόζουμε τα ήδη γνωστά στατιστικά για το "standard" ή "minmax" scaling και γράφουμε τα νέα, κανονικοποιημένα δεδομένα στο αρχείο εξόδου (row-major order). Έχουν προστεθεί έλεγχοι για την **αποφυγή διαίρεσης με το μηδέν** (τόσο αν $\sigma_j = 0$ όσο και αν $\max_j = \min_j$).

Out-of-Core Επεξεργασία (Μεγάλα Αρχεία): Ο κώδικας δεν φορτώνει ολόκληρο το αρχείο στη μνήμη. Δεσμεύει μνήμη μόνο

για έναν buffer μεγέθους `block_rows * D` στοιχείων. Το αρχείο διαβάζεται τμηματικά (blocks), το κάθε block επεξεργάζεται in-place και η εφαρμογή προχωράει, κρατώντας το memory footprint σε εξαιρετικά χαμηλά επίπεδα.

2. Αναλυτική Τεκμηρίωση Παράλληλων Υλοποιήσεων & Βελτιστοποιήσεων

Η αρχιτεκτονική του συστήματος βασίστηκε στην επεξεργασία δύο φάσεων. Για την αποφυγή υπερχείλισης της μνήμης (Out-of-Core), ορίστηκε εξ ορισμού η παράμετρος `block_rows = 100000`, διαβάζοντας τα δεδομένα σε "κομμάτια".

2.1 Σειριακή Υλοποίηση Αναφοράς (serial_scaler.c)

Η σειριακή έκδοση αποτελεί τη βάση για τον έλεγχο ορθότητας και τον υπολογισμό του speedup. Ικανοποιεί τα ζητούμενα ως εξής:

- Ανάγνωση / Εγγραφή raw binary αρχείου: Πραγματοποιείται με τις τυπικές συναρτήσεις της C `fread()` και `fwrite()`, χωρίς αναμονή μεταδεδομένων (τα N και D περνιούνται μέσω `argv`).
- Υπολογισμός στατιστικών & Εφαρμογή Scalers: Το πρόγραμμα διατρέχει τον πίνακα buffer με διπλό βρόχο (outer loop γραμμών, inner loop στηλών). Στη 2η φάση ελέγχει τη μεταβλητή mode για να εφαρμόσει τον αντίστοιχο τύπο μετασχηματισμού.
- Επεξεργασία με blocks: Χρησιμοποιείται ένας βρόχος `while (rows_processed < N)` που διαβάζει κάθε φορά το πολύ `block_rows` γραμμές, διασφαλίζοντας ότι η διαθέσιμη RAM δεν υπερφορτώνεται.

2.2 SIMD Υλοποίηση (simd_scaler.c)

Η SIMD έκδοση αξιοποιεί τους καταχωρητές των 256-bit (AVX2) για την επιτάχυνση των πράξεων της CPU σε επίπεδο εντολών.

- Τμήματα που έγιναν vectorized: Ο εσωτερικός βρόχος των στηλών (j) τόσο στη Φάση 1 (ενημέρωση αθροισμάτων και min/max) όσο και στη Φάση 2 (πράξεις μετασχηματισμού in-place).
- Μέθοδος Vectorization: Χρησιμοποιήθηκαν explicit SIMD intrinsics της βιβλιοθήκης <immintrin.h>. Γράψαμε τον βρόχο να προχωράει ανά 4 διπλής ακρίβειας στοιχεία. Δεδομένα φορτώνονται ταυτόχρονα μέσω της `_mm256_loadu_pd()`, γίνονται πράξεις (`_mm256_add_pd()`) και αποθηκεύονται με `_mm256_storeu_pd()`. Για τις στήλες που περισσεύουν, υλοποιήθηκε ένα "Remainder Loop".
- Βελτιστοποίηση Διαίρεσης: Στη Φάση 2 εφαρμόσαμε τη μετατροπή της αργής διαίρεσης σε ταχύτατο πολλαπλασιασμό με τον αντίστροφο (`1.0 / std_v[j]`), αξιοποιώντας την `_mm256_mul_pd()`.
- Compiler Flags: Χρησιμοποιήθηκε το flag `-mavx2` σε συνδυασμό με το `-O3`.
- Απόδοση & Bottlenecks: Η υλοποίηση βελτίωσε αισθητά την απόδοση, μειώνοντας τον καθαρό υπολογιστικό χρόνο (user time) σχεδόν στο μηδέν. Ο συνολικός χρόνος περιορίστηκε αποκλειστικά από την ταχύτητα ανάγνωσης/εγγραφής του δίσκου (I/O Bound).

2.3 OpenMP Υλοποίηση (omp_scaler.c)

Στην υλοποίηση με OpenMP, το ζητούμενο ήταν η κατανομή του υπολογιστικού φόρτου στους διαθέσιμους πυρήνες.

- Κατανομή εργασίας: Στις Φάσεις 1 και 2 χρησιμοποιήθηκε η οδηγία `#pragma omp parallel for` στο εξωτερικό loop των γραμμών (i). Έτσι, τα threads διαβάζουν συνεχόμενα δεδομένα στη μνήμη (row-major order), αξιοποιώντας τη CPU Cache.
- Στρατηγική αποφυγής race conditions: Για την αποφυγή συγκρούσεων στις ίδιες στήλες, **δεν** χρησιμοποιήθηκαν αργές πράξεις `#pragma omp critical` ή `atomic`.

Αντιθέτως, έγινε χρήση των σύγχρονων Array Reductions του OpenMP 4.5+ (`reduction(+:sum_v[:D])`). Το OpenMP δημιουργεί αυτόματα ιδιωτικούς πίνακες για κάθε thread και τους συγχωνεύει ταχύτατα στο τέλος, εξασφαλίζοντας 100% ασφάλεια.

- Επίδραση του block size: Η χρήση blocks των 100.000 γραμμών βοήθησε ώστε τα δεδομένα να χωράνε στην L3 Cache, διατηρώντας το RAM footprint χαμηλό.
- Συμπεριφορά Threads: Η αύξηση των threads (1 έως 16) δεν οδήγησε σε ουσιαστική μείωση του χρόνου. Επειδή οι εντολές fread/fwrite εκτελούνται σειριακά εκτός της παράλληλης περιοχής, τα υπόλοιπα threads έκαναν «busy-waiting» (spinning) περιμένοντας τον δίσκο, με αποτέλεσμα την εκτόξευση του user time (Νόμος του Amdahl).

2.4 MPI Υλοποίηση (mpi_scaler.c)

Η έκδοση MPI κατανέμει τον φόρτο σε πολλαπλές ανεξάρτητες διεργασίες.

- Κατανομή γραμμών: Ο συνολικός αριθμός N διαιρείται ισόποσα με το μέγεθος (size) των διεργασιών. Το υπόλοιπο της διαίρεσης μοιράζεται δίκαια στα πρώτα ranks (Load Balancing). Κάθε διεργασία υπολογίζει το δικό της offset γραμμών (start_row).
- Είδος I/O: Για την εξάλειψη του bottleneck, υλοποιήσαμε Παράλληλο Κλασικό File I/O. Κάθε rank ανοίγει το αρχείο, υπολογίζει το byte offset του και μετακινεί τον pointer με τη συνάρτηση fseeko(). Στη Φάση 2, το αρχείο εξόδου ανοίγει με "r+b", επιτρέποντας την ταυτόχρονη εγγραφή σε διαφορετικά σημεία χωρίς να καταστρέφονται τα δεδομένα.
- Συνολικά στατιστικά & Κόστος: Μετά τον υπολογισμό των τοπικών πινάκων, γίνεται χρήση της συλλογικής επικοινωνίας MPI_Allreduce (MPI_SUM, MPI_MIN, MPI_MAX) ώστε όλες οι διεργασίες να λάβουν τα τελικά

στατιστικά. Το κόστος αυτών των επικοινωνιών είναι πρακτικά μηδενικό (μεταφορά μόλις D doubles).

- Bottleneck στο Filesystem: Η ικανότητα του λειτουργικού να εξυπηρετεί παράλληλα ανεξάρτητους pointers I/O (disk striping) εξάλειψε το bottleneck του δίσκου, αποδίδοντας την κορυφαία επιτάχυνση στα πειράματά μας.

2.5 CUDA Υλοποίηση (cuda_scaler.cu)

Η υλοποίηση στοχεύει στην αποφόρτιση των μαθηματικών υπολογισμών στην NVIDIA Tesla V100 GPU.

- Διαχωρισμός CPU-GPU: Η CPU κρατά τον ρόλο του "I/O Manager" (διαβάζει με fread) και υπολογίζει τα τελικά στατιστικά (std_v). Η GPU αναλαμβάνει τις πράξεις των εκατομμυρίων στοιχείων (min, max, scaling) μέσω των compute_stats_kernel και scale_kernel.
- Εκτέλεση Kernels & Atomics: Στη GPU χρησιμοποιήθηκε εγγενές atomicAdd για double. Αντίθετα, για το min/max υλοποιήθηκαν **custom atomic συναρτήσεις** (atomicMinDouble, atomicMaxDouble) με χρήση atomicCAS. Λόγω των πολλαπλών ταυτόχρονων εγγραφών, παρατηρείται atomic contention.
- Κόστος Μεταφοράς (CPU-GPU / GPU-CPU): Καθώς το dataset δεν χωράει στη VRAM (Out-of-Core), κάθε block αντιγράφεται συνεχώς μπρος-πίσω (cudaMemcpyHostToDevice και cudaMemcpyDeviceToHost). Αυτό το τεράστιο κόστος μεταφοράς μέσω του διαύλου PCIe αποτελεί το **κύριο bottleneck** της υλοποίησης, το οποίο επισκιάζει τη μαθηματική ταχύτητα της GPU.

3. Πειραματική Αξιολόγηση και Αποτελέσματα

3.1 Έλεγχος Ορθότητας & Σφάλματα (Absolute Errors)

Όλες οι υλοποιήσεις ελέγχθηκαν συγκρίνοντας το παραγόμενο αρχείο με τα αποτελέσματα της βιβλιοθήκης scikit-learn (Python). Όπως προκύπτει, το Μέγιστο Απόλυτο Σφάλμα (Max

Absolute Error) είναι πρακτικά μηδενικό (της τάξης του $10^{(-13)}$ έως $10^{(-16)}$). Οι απειροελάχιστες διαφορές ανάμεσα στις παράλληλες εκδόσεις οφείλονται αποκλειστικά στη μη-προσεταριστικότητα της πρόσθεσης κινητής υποδιαστολής (τα threads/ranks προσθέτουν τα νούμερα με διαφορετική σειρά).

Όλες οι δοκιμές επέστρεψαν SUCCESS.

1. Small Dataset (N = 1.000.000, D = 32)

Standard Scaler

Υλοποίηση	Max Absolute Error	Mean Absolute Error	Αποτέλεσμα
Serial	1.9007018182e-13	8.8254248799e-15	SUCCESS
SIMD (AVX2)	1.9007018182e-13	8.8301304266e-15	SUCCESS
OpenMP (16 Threads)	2.0428103653e-14	1.5254252783e-15	SUCCESS
MPI (16 Procs)	2.3980817332e-14	1.6501411465e-15	SUCCESS
CUDA (GPU)	1.9717560917e-13	8.8944602873e-15	SUCCESS

MinMax Scaler

Υλοποίηση	Max Absolute Error	Mean Absolute Error	Αποτέλεσμα
Serial	2.2204460493e-16	3.3937129128e-17	SUCCESS
SIMD (AVX2)	2.2204460493e-16	2.6937085768e-17	SUCCESS

OpenMP (16 Threads)	2.2204460493e-16	2.6937085768e-17	SUCCESS
MPI (16 Procs)	2.2204460493e-16	2.6937085768e-17	SUCCESS
CUDA (GPU)	2.2204460493e-16	2.6937085768e-17	SUCCESS

2. Medium Dataset (N = 5.000.000, D = 64)

Standard Scaler

Υλοποίηση	Max Absolute Error	Mean Absolute Error	Αποτέλεσμα
Serial	6.6613381478e-13	1.8453982406e-14	SUCCESS
SIMD (AVX2)	6.6702199319e-13	1.8457143850e-14	SUCCESS
OpenMP (16 Threads)	2.2204460493e-14	7.4166493719e-16	SUCCESS
MPI (16 Procs)	3.8191672047e-14	1.8022071856e-15	SUCCESS
CUDA (GPU)	6.6346927952e-13	1.8435131746e-14	SUCCESS

MinMax Scaler

Υλοποίηση	Max Absolute Error	Mean Absolute Error	Αποτέλεσμα
Serial	3.3306690739e-16	3.5438652502e-17	SUCCESS
SIMD (AVX2)	2.2204460493e-16	2.9281379583e-17	SUCCESS

OpenMP (16 Threads)	2.2204460493e-16	2.9281379583e-17	SUCCESS
MPI (16 Procs)	2.2204460493e-16	2.9281379583e-17	SUCCESS
CUDA (GPU)	2.2204460493e-16	2.9281379583e-17	SUCCESS

3. Large Dataset (N = 10.000.000, D = 128)

Standard Scaler

Υλοποίηση	Max Absolute Error	Mean Absolute Error	Αποτέλεσμα
Serial	1.3375967001e-12	2.7795262839e-14	SUCCESS
SIMD (AVX2)	1.3375967001e-12	2.7800795998e-14	SUCCESS
OpenMP (16 Threads)	1.4210854715e-14	6.0036047921e-16	SUCCESS
MPI (16 Procs)	4.7961634664e-14	1.9416990149e-15	SUCCESS
CUDA (GPU)	1.3402612353e-12	2.7814327204e-14	SUCCESS

MinMax Scaler

Υλοποίηση	Max Absolute Error	Mean Absolute Error	Αποτέλεσμα
Serial	2.2204460493e-16	3.5782645376e-17	SUCCESS
SIMD (AVX2)	2.2204460493e-16	2.8288217494e-17	SUCCESS

OpenMP (16 Threads)	2.2204460493e-16	2.8288217494e-17	SUCCESS
MPI (16 Procs)	2.2204460493e-16	2.8288217494e-17	SUCCESS
CUDA (GPU)	2.2204460493e-16	2.8288217494e-17	SUCCESS

3.2 Χρόνοι Εκτέλεσης και Επιτάχυνση (Speedup)

Το Speedup υπολογίζεται ως $T_{\text{serial}} / T_{\text{parallel}}$.

1. Small Dataset (N=1.000.000, D=32)

Το αρχείο είναι μικρό (~256 MB) και χωράει ολόκληρο στη μνήμη του συστήματος.

Standard Scaler

Υλοποίηση	Χρόνος(sec)	Speedup
Serial	1.60	1.00x
SIMD (AVX2)	0.58	2.76x
OpenMP (16 Threads)	1.15	1.39x
MPI (16 Procs)	0.96	1.67x
CUDA (GPU)	2.53	0.63x

MinMax Scaler

Υλοποίηση	Χρόνος(sec)	Speedup
Serial	2.99	1.00x

SIMD (AVX2)	1.38	2.17x
OpenMP (16 Threads)	1.66	1.80x
MPI (16 Procs)	1.04	2.88x
CUDA (GPU)	1.28	2.34x

2. Medium Dataset (N=5.000.000, D=64)

Μεσαίο αρχείο (~2.56 GB).

Standard Scaler

Υλοποίηση	Χρόνος (sec)	Speedup
Serial	28.55	1.00x
SIMD (AVX2)	19.47	1.47x
OpenMP (16 Threads)	19.74	1.45x
MPI (16 Procs)	3.81	7.49x
CUDA (GPU)	21.80	1.31x

MinMax Scaler

Υλοποίηση	Χρόνος (sec)	Speedup
Serial	32.62	1.00x
SIMD (AVX2)	16.29	2.00x
OpenMP (16 Threads)	17.81	1.83x
MPI (16 Procs)	3.94	8.28x
CUDA (GPU)	17.63	1.85x

Παρακάτω παρατίθενται οι επιδόσεις στο "ακραίο" Large Dataset (~10.24 GB), το οποίο αναδεικνύει την πραγματική Out-of-Core συμπεριφορά του I/O συστήματος.

Standard Scaler (Large Dataset)

Υλοποίηση	Χρόνος(sec)	Speedup
Serial	107.22	1.00x
SIMD (AVX2)	72.55	1.48x
OpenMP (16 Threads)	73.71	1.45x
MPI (16 Procs)	12.85	8.34x
CUDA (NVIDIA V100)	88.29	1.21x

MinMax Scaler (Large Dataset)

Υλοποίηση	Χρόνος(sec)	Speedup
Serial	85.94	1.00x
SIMD (AVX2)	73.00	1.18x
OpenMP (16 Threads)	86.89	0.99x (Στάσιμο)
MPI (16 Procs)	12.11	7.10x
CUDA (NVIDIA V100)	91.22	0.94x (Στάσιμο)

4. ΣΥΜΠΕΡΑΣΜΑΤΑ

1. Η απόλυτη υπεροχή του MPI (Speedup ~8.3x): Ο σχεδιασμός που χρησιμοποιήθηκε στο MPI επέτρεψε την ανεξάρτητη και παράλληλη ανάγνωση του αρχείου από πολλαπλές διεργασίες μέσω της `fseeko()`. Το λειτουργικό σύστημα του Linux αντέδρασε εξαιρετικά

(παράλληλο I/O), με αποτέλεσμα να εξαλειφθεί πλήρως το bottleneck του σκληρού δίσκου και ο χρόνος να πέσει δραματικά στα 12 δευτερόλεπτα.

2. Ο Νόμος του Amdahl στο OpenMP (Στασιμότητα): Η αδυναμία του OpenMP να ξεπεράσει το 1.45x speedup (ακόμη και με 16 πυρήνες) εξηγείται πλήρως. Επειδή η ανάγνωση της μνήμης γινόταν σειριακά (1 master thread με fread), ο συνολικός χρόνος εκτέλεσης περιορίστηκε από τον χρόνο ανάγνωσης/εγγραφής. Τα υπόλοιπα threads "σπίναραν" σε busy-waiting, εκτοξεύοντας τον χρήσιμο χρόνο (user time) της CPU στα 81 δευτερόλεπτα χωρίς πρακτικό κέρδος σε real time.
3. Το Bottleneck του PCIe (CUDA): Παρά τη χρήση της πανίσχυρης V100, η επεξεργασία του 10GB αρχείου ανάγκασε την εφαρμογή να αντιγράφει ασταμάτητα τα δεδομένα μέσω του διαύλου PCIe. Αυτός ο σχεδιασμός out-of-core με pageable memory ακύρωσε ολοκληρωτικά τη μαθηματική ταχύτητα της GPU, οδηγώντας σε ταχύτητες παρόμοιες της αρχικής σειριακής υλοποίησης, επιβεβαιώνοντας ότι οι GPUs αποδίδουν ιδανικά όταν τα δεδομένα βρίσκονται ήδη στην τοπική τους μνήμη (VRAM).
4. SIMD Efficiency: Η SIMD υλοποίηση επέδειξε "δωρεάν" επιτάχυνση (περίπου ~1.5x) σε σχέση με τον σειριακό κώδικα, μειώνοντας δραστικά τους κύκλους ρολογιού χωρίς το παραμικρό overhead διαχείρισης threads ή δικτύου.